# CS 3502 Project 3 Report

File System Implementation

Diego Urena

Spring 2026

## 1 Introduction

This project implements a desktop file manager for CS 3502 Project 3 using C# and Avalonia UI. The goal is to connect operating-system file system concepts (files, directories, permissions, metadata, and error handling) with a user-facing GUI that supports CRUD operations and directory navigation.

### Technologies Chosen

- **Language:** C# (.NET 10)
- **GUI Framework:** Avalonia (cross-platform desktop UI)
- **Core APIs:** `System.IO` for file/directory operations, platform-specific APIs for permissions

Avalonia was chosen because it supports Linux, Windows, and macOS with one UI codebase. This aligns with the project's focus on OS-level behavior while keeping the GUI portable.

## 2 Operating Systems Concepts

### System Calls and Abstractions

The application uses high-level C# APIs that eventually invoke OS system calls:

- `File.ReadAllText`, `File.WriteAllText` → open/read/write/close style operations
- `Directory.GetFiles`, `Directory.GetDirectories` → directory traversal calls
- `File.Move`, `Directory.Move` → rename/move semantics
- `File.Delete`, `Directory.Delete` → unlink/rmdir style operations
- `File.GetLastWriteTime`, `File.GetCreationTime` → metadata/stat-like queries

The GUI abstracts these operations by mapping user actions (toolbar clicks, tree selection, double-click to open file) to service methods in `FileOperationsService`.

### File Descriptors and Inodes in Context

A file descriptor is the process-local handle returned by the OS when a file is opened. In this project, descriptor management is mostly implicit through .NET APIs and explicit when creating files with `FileStream`. Using statements and API-level resource handling ensure handles are closed after use.

Metadata such as size/time/permissions originates from inode-like filesystem metadata structures (or NTFS equivalents on Windows). The app displays timestamps and permissions in the file information panel, and reloads metadata on selection changes.

## 3 Design and Architecture

### High-Level Structure

The project follows separation of concerns:

- `Views/*`: Avalonia windows and dialogs
- `Services/FileOperationsService.cs`: file/folder CRUD, move/copy/paste, validation, error wrapping
- `Services/RootTreeService.cs`: root tree initialization from environment
- `Services/TreeViewStateService.cs`: expanded/selected tree state persistence on refresh
- `Models/ListableItem.cs`: tree node model with metadata, permissions, children, and reload logic
- `Providers/Permissions/*`: OS-specific permission retrieval via a factory

TODO: Insert architecture diagram screenshot here.
(Replace this box with `\includegraphics`)

Figure 1: Architecture overview (GUI, services, model, and permissions providers).

### Design Decisions

- UI event handlers in `MainWindow` call service-layer methods instead of embedding file-system logic directly in the view.
- A dedicated `FileOperationResult` object standardizes success/failure reporting and root path updates after rename/move.
- Permission logic is delegated to `UnixPermissionsProvider` and `WindowsPermissionProvider`, chosen by `PermissionsProviderFactory`.
- Tree refresh preserves selected and expanded state through `TreeViewStateService`.

# 4 Implementation

## Core Operations

All required core operations are implemented:

- **Create**: Add file and add folder via validated input dialogs.
- **Read**: Open file viewer on double-click, load content from disk.
- **Update**: Edit file content in viewer and save changes.
- **Delete**: Confirmation dialog before deleting files/folders.
- **Rename**: Dialog-based rename for files and folders.
- **Navigate**: Tree view directory traversal with expand/collapse and refresh tools.

## Representative File Operation Snippet

Listing 1: Simplified pattern used by FileOperationsService for robust operations

```
private static FileOperationResult Execute(Func<FileOperationResult>
    operation)
{
    try
    {
        return operation();
    }
    catch (Exception ex) when (ex is IOException or
        UnauthorizedAccessException
                                or ArgumentException or
                                    NotSupportedException)
    {
        return FileOperationResult.Fail(ex.Message);
    }
}
```

## GUI Integration

Main UI features include:

- Left tree panel and right file-information panel split with a grid splitter.
- Toolbar actions for copy/cut/paste, add file/folder, rename, move, delete, refresh, expand, collapse.
- Search box for filtering tree items by name or path.
- Light/dark mode toggle.
- Keyboard shortcuts: Ctrl+C, Ctrl+X, Ctrl+V.

TODO: Insert main window screenshot showing tree, toolbar, and file info panel.

Figure 2: Main file manager window.

TODO: Insert file viewer screenshot (editable and read-only behavior).

Figure 3: File viewer window.

### Error Handling Strategy

The application catches common file-system exceptions and reports user-visible messages through dialogs. Validation methods prevent invalid names, duplicate names, invalid destinations, and unsafe moves (e.g., moving a folder into itself). Permission checks are included in metadata/permissions display and edit-mode enablement in file viewer.

## 5 Testing

### Required Scenarios and Status

The following list maps required functionality and guideline items to the current codebase state.

- **Create (file/folder):** Met. Evidence: `OnAddFileClicked`, `OnAddFolderClicked`, `FileOperationsService`
- **Read (view content):** Met. Evidence: double-click in tree opens `FileViewer`; read in `FileViewer` constructor.
- **Update (edit/save):** Met. Evidence: `FileViewer.SaveFile` writes updates; unsaved-change confirmation on close.
- **Delete (with confirmation):** Met. Evidence: `ConfirmationDialog` plus `FileOperationsService.Delete`
- **Rename:** Met. Evidence: `OnRenameClicked` plus `FileOperationsService.Rename`.
- **Navigate tree:** Met. Evidence: recursive `ListableItem.Children` with expand/collapse support.
- **Minimum GUI components:** Mostly met. File display, action buttons, file content area, and feedback dialogs exist. A persistent status bar is not explicit.
- **Current path display:** Partial. Selected item path is shown, but an always-visible breadcrumb/current-folder display is not explicit.
- **Permission handling:** Mostly met. OS-specific providers plus read-only viewer mode and exception handling.
- **Atomic rename:** Met. Uses `File.Move/Directory.Move` semantics.
- **Cross-volume move fallback:** Partial. No copy+delete fallback path for cross-device moves.
- **Error normalization by code:** Partial. Exceptions are handled but user messages are mostly raw exception text.

- **Separation of concerns:** Met. View, service, and provider responsibilities are separated.
- **Optional enhancements implemented:** Search and keyboard shortcuts are implemented; context menu was replaced by a toolbar workflow.

### Manual Test Coverage Notes

The workspace includes a permissions/concurrency test area under `ExamplesRoot/permsTest` and related directories. These are appropriate for testing:

- permission denied scenarios,
- read-only behavior,
- concurrent access edge cases,
- invalid destination and duplicate-name handling.

TODO: Insert screenshot of permission-denied or file-in-use error dialog.

Figure 4: Example graceful error handling case.

## 6 Reflection Questions

### 1) System Call Translation

A high-level create operation in C# (for example `new FileStream(path, FileMode.CreateNew, ...)`) maps to a sequence similar to open/create, write, and close at the OS boundary. The runtime marshals managed calls into native OS calls. The file system then updates metadata and directory entries.

### 2) File Descriptors

A file descriptor is an index into the process file table that references an open file object in the kernel. On open, the OS verifies path and permissions and allocates descriptor state. Reads use that descriptor to fetch bytes and update offsets. Close releases descriptor resources. If many files are opened without closing, the process can hit descriptor limits and fail with "too many open files" errors.

### 3) Inodes and Metadata

Displayed metadata (timestamps, size, permissions) comes from filesystem metadata objects (inode-like entries on Unix, MFT/security descriptors on NTFS). The API-level equivalent is stat-like metadata queries. Filenames are stored in directory entries mapping names to inode numbers, not inside the inode itself.

### 4) Directory Traversal

Directories store mappings from names to file metadata references. "Opening" a directory allows iteration of entries (through APIs such as `Directory.GetFiles/GetDirectories`). Path resolution proceeds component by component from root/current directory until the target object is found or resolution fails.

### 5) Permission Denied

The OS checks permissions during the attempted operation (open, delete, write, etc.). Deletion rights typically depend on directory permissions and ownership rules, not only file-level flags. The application detects failures through exceptions (for example `UnauthorizedAccessException`) and surfaces user-facing feedback dialogs.

### 6) Atomicity

Rename is typically atomic within one filesystem because directory-entry updates occur as a single filesystem transaction. Without atomicity, partial states could expose both old/new names or neither. In the current implementation, rename failures return an error result and do not apply UI refresh as success.

### 7) Buffering

Buffering occurs at both application/runtime and kernel levels. Writes may appear successful before media flush. If a program crashes before buffers flush, some data can be lost unless explicit flush/sync behavior is used.

### 8) File Manager Comparison

Professional managers provide richer features: thumbnails, drag-and-drop, trash/recycle integration, shell extensions, network share handling, indexing, ACL editors, and background file transfer queues. Implementing these requires additional OS integrations, async job systems, and more complex state management. The trade-off is significantly higher complexity, testing burden, and platform-specific code.

## 7 Challenges and Solutions

- **Cross-platform permissions**: solved with provider abstraction and factory selection by host OS.
- **Tree refresh state loss**: solved using `TreeViewStateService` for expanded/selected path restoration.
- **Safe destructive operations**: solved using explicit confirmation dialogs and centralized operation wrappers.
- **UI/logic separation**: solved by keeping file-system actions in `FileOperationsService` and keeping view handlers thin.

## 8 Conclusion

The project successfully implements the required CRUD and navigation capabilities with a structured architecture and cross-platform GUI. It demonstrates OS concepts including metadata retrieval, permission checks, resource handling, and user-facing error management.

### Remaining Work Before Final Submission

Based on current code and project requirements, the highest-value remaining improvements are:

1. Add a persistent **status/feedback area** in the main window (not only dialogs).
2. Add an always-visible **current path display** (breadcrumb or current-folder label).
3. Improve **error normalization** by mapping common exceptions/error codes to consistent, friendly messages.
4. Add optional **cross-volume move fallback** (copy+delete with rollback/error checks).
5. Ensure submission packaging includes **README build/run instructions** and dependency list.